OpenTelemetry in .NET

What Tracing, Metrics, and automatic correlation add on top of structured logging.

Source code available after this talk.

Nine Questions, in the Order You'd Actually Hit Them

Topic	What you'll be able to do after
Introduction	Tell Traces, Metrics and Logs apart, and explain why they need a shared Context
Manual Instrumentation	Register a Resource and Instrumentation Libraries, in one call
Tracing	Configure Tracing, export to Jaeger, and enrich a Span with tags
Metrics	Pick Counter, Histogram or ObservableGauge, and export to Prometheus
OpenTelemetry Collector	Decide when you need one, and configure the pipeline that fans out
Logging	Export Logs to Loki and enrich them with the same data as a Span
Context Propagation	Carry context over RabbitMQ and Baggage, and know the PII risk
Deep Diving into Tracing	Write a Span Processor, create Spans by hand, record an Exception
Sampling	Explain Head vs. Tail Sampling, and write a custom Sampler

Every example on the following slides is real code, from one of two running projects: **OrderPipeline** (5 services, RabbitMQ, a Saga) or **Bookmarks** (one service, no broker).

Monitoring Tells You Something's Wrong. Observability Tells You Why.

A dashboard built from Serilog and CloudWatch Logs Insights can tell you *that* checkout got slower at 2pm. It was built to answer the questions someone thought to ask in advance. **Observability** is the property of a system where you can answer a question nobody wrote a dashboard for, by exploring the data you already collected.

The question	Structured logging alone	OpenTelemetry
Which of our 5 services was slow?	Grep timestamps across 5 log streams and guess	One Trace, one correlated tree of Spans
Is this request typical, or an outlier?	Scan log lines by eye	A Histogram already computed p50/p95/p99
Which log line belongs to which request?	A correlation id you thread through every call by hand	A TraceId attached automatically

OpenTelemetry is a vendor-neutral standard, an API, SDK and wire protocol, for producing these three kinds of telemetry the same way regardless of which backend eventually reads them.

Three Signals, Three Different Questions

OpenTelemetry standardizes three kinds of telemetry. None of them replaces another, each answers something the others can't.

Signal	What it is	The question it answers
Trace	A tree of Spans, one per unit of work, across services	What happened, in what order, and where did the time go, for this one request?
Metric	A number, aggregated at the source, over time	Is this normal, and is it getting better or worse, across every request?
Log	A timestamped, free-form or structured event	What exactly did the code say happened, in its own words?

A fourth concept, **Baggage**, isn't a signal. It's key-value context that rides along with a request and gets copied onto whichever Spans and Logs choose to read it. Context Propagation, later in this talk, covers it.

Three Ids Are What Make "Correlation" Possible

Every Span is identified by a TraceId and a SpanId. A child Span also records its ParentSpanId. That's the entire mechanism: no central registry, no database lookup, the tree rebuilds itself from these three fields.

Real console output, Bookmarks.Api, one GET /bookmarks call

```
Activity.TraceId: c071e36adec84e2b2c452bbc0dc30ea7
Activity.SpanId: 84113f521a72f0b5
Activity.DisplayName: GET bookmarks
Activity.Kind: Server
```

That same TraceId is what a Log line carries, what a Metric's exemplar can point back to, and what you'd paste into a support ticket. It's the one value that ties all three signals, and every Span in the tree, to the same request.

OpenTelemetry Didn't Add a New API. .NET Already Had One.

System.Diagnostics.Activity and ActivitySource have existed in the base class library since .NET 5, before this talk's OpenTelemetry stack existed. An Activity *is* a Span. OpenTelemetry's SDK is a listener that subscribes to the sources you name and exports what it hears.

The identical 4-line pattern, in two unrelated projects:

Payment.Api/Diagnostics/ApplicationDiagnostics.cs

```
public static class ApplicationDiagnostics
{
  public static readonly ActivitySource
    ActivitySource = new("Payment.Api");
}
```

Bookmarks.Api/Diagnostics/ApplicationDiagnostics.cs

```
public static class ApplicationDiagnostics
{
  public static readonly ActivitySource
    ActivitySource = new(ServiceName);
}
```

One service is a 5-process distributed Saga, the other is a single REST API with no broker. The instrumentation code doesn't know, or care, which one it's running in.

A Span Is Meaningless Until You Say Whose It Is

A **Resource** is the set of attributes stamped on every Span, Metric and Log a process emits: which service, which version, which environment. Without it, a backend holding telemetry from 5 services can't tell them apart.

Bookmarks.Api/appsettings.json, read by ObservabilityResourceFactory.Configure

```
"Observability": {
  "ServiceName": "Bookmarks.Api",
  "ServiceNamespace": "Bookmarks",
  "ResourceAttributes": { "team": "PersonalPractice" }
}
```

The same service.namespace convention is what lets OrderPipeline's 5 .NET services and its Next.js front end, Web.BFF, show up under one namespace in Jaeger, even though one side is C# and the other is TypeScript:

```
// web/src/instrumentation.ts, @vercel/otel
attributes: { "service.namespace": "OrderPipeline", team: "PersonalPractice" }
```

Automatic Instrumentation Isn't Magic, It's Subscription

An **Instrumentation Library** is code someone else wrote that calls ActivitySource.StartActivity on your behalf, at the framework's call sites. Two different ways to turn one on, both real:

A NuGet package you add, Payment.Api

```
tracing.AddNpgsql();
```

One extension method. Needs its own package reference per service.

Just naming its ActivitySource, Bookmarks.Api

```
observability.ActivitySources.Add("Npgsql");
```

No Npgsql-specific package at all. The driver already publishes its own source, naming it is enough.

Either way, the result is the same 3-Span tree, with no code written at the call site:

```
GET bookmarks Server, automatic, ASP.NET Core instrumentation
└── Bookmarks.List Internal, manual, written by hand
    └── postgresql Client, automatic, the Npgsql driver's own source
```

One Builder, Five Decisions

Configuring Tracing means answering, in order: which Sampler, which Instrumentation Libraries, which hand-written ActivitySources, does Baggage get copied onto Spans, and where does it export to.

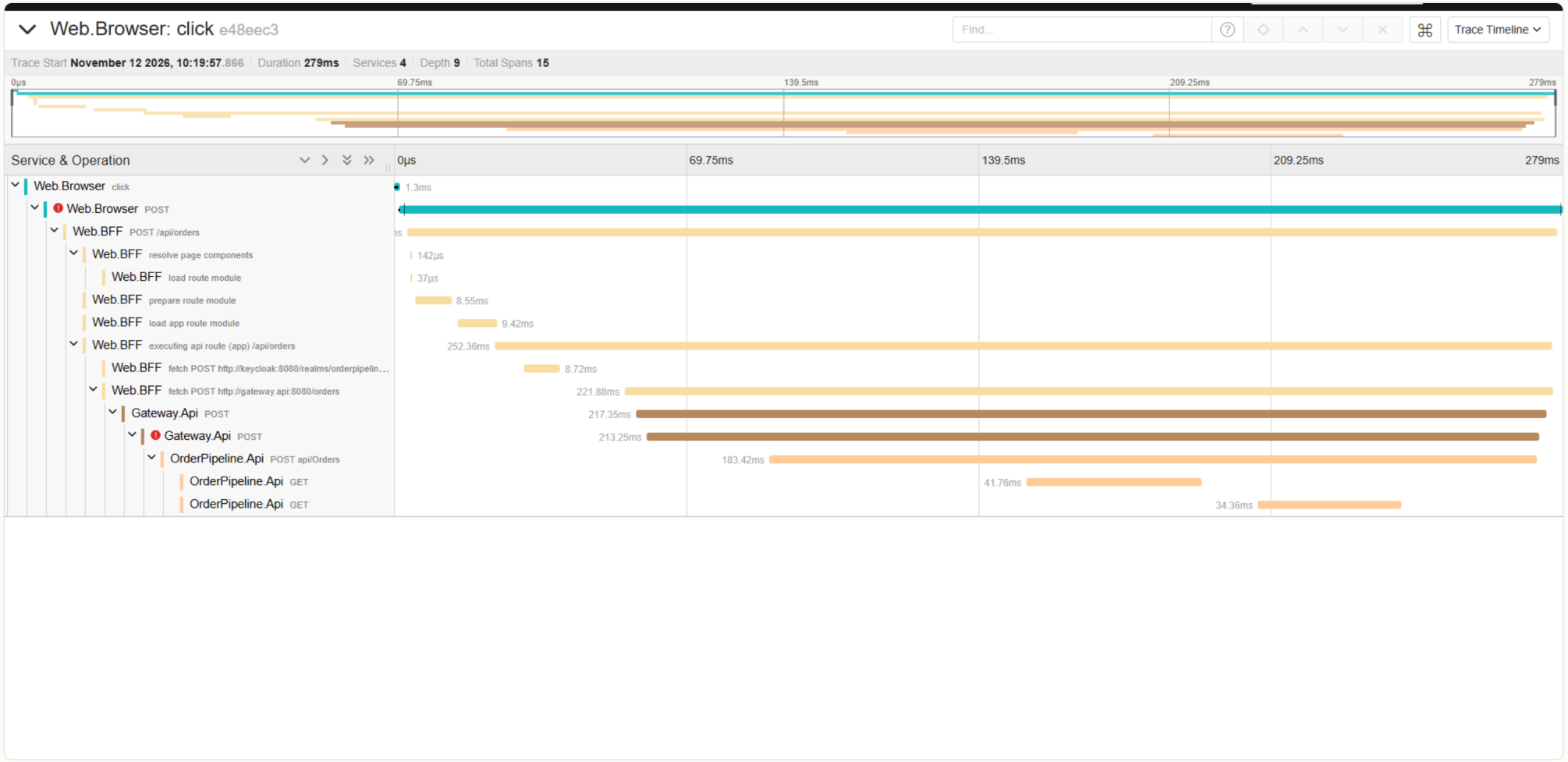
Shared.Observability/Internal/TracingConfigurator.cs, vendored unchanged in both projects

```
public static void Configure(TracerProviderBuilder tracing, ObservabilityOptions options)
{
  AddSampler(tracing, options);
  AddInstrumentation(tracing, options.Instrumentation);
  foreach (var source in options.ActivitySources) tracing.AddSource(source);
  if (options.Baggage.AllowedKeyPrefixes.Count > 0)
    tracing.AddProcessor(new BaggageTagProcessor(options.Baggage));
  AddExporters(tracing, options);
}
```

Every service that calls builder.AddSharedObservability(...) runs this exact method. OrderPipeline.Api and Bookmarks.Api differ only in which options they pass in.

One Click, Four Services, Zero Manual Correlation Code

A real "Place order" click from OrderPipeline's store. @vercel/otel in the browser, Next.js's own instrumentation, and AddAspNetCoreInstrumentation in both .NET services, hand the same traceparent header forward automatically.



This is the HTTP path only, Browser through to OrderPipeline.Api. What happens next, across RabbitMQ, is the subject of Context Propagation later in this talk, and it does not stay this clean.

A Tag Turns a Span Into a Decision You Can Query

A Span with just a name and a duration tells you something was slow. A Span with tags tells you *which order*, *which outcome*, so you can filter and group by them in the backend, not just read one at a time.

Payment.Api/Messaging/StockReservedHandler.cs

```
using var activity = ApplicationDiagnostics.ActivitySource.StartActivity("authorize payment");
activity?.SetTag(OrderPipelineTags.OrderId, @event.OrderId);
activity?.SetTag(OrderPipelineTags.OrderTotalAmount, @event.Amount);
⋮
activity?.SetTag(OrderPipelineTags.Outcome, OrderPipelineTags.OutcomeFailed);
```

In Bookmarks.Api the same pattern enriches a different kind of Span, after the database write succeeds:

```
operation.Activity?.SetTag(BookmarkTags.BookmarkId, bookmark.Id);
```

The ?. matters: a Sampler can decide not to record this Span at all, and the call must still be safe.

A Metric Is a Number That's Already Cheap by Export Time

Mining log lines for a trend means scanning every line, every time, at query time. A Meter aggregates at the source instead, so the backend stores a handful of numbers, not every request.

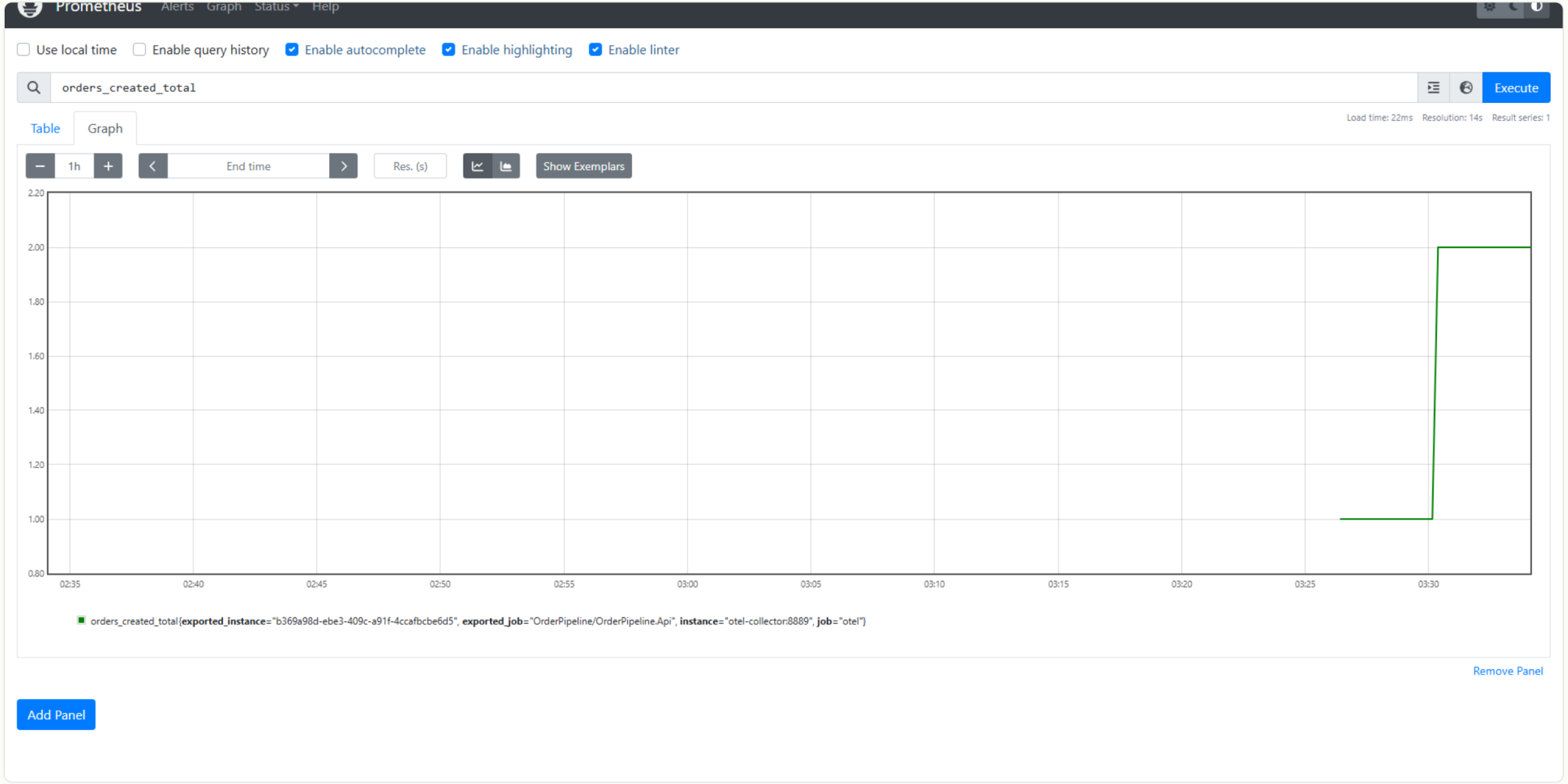
Instrument	Behavior	Real example
Counter<T>	Only ever increases	bookmarks.created, Bookmarks.Api
Histogram<T>	Value distribution (p50/p95/p99)	bookmarks.operation.duration, tagged by operation
ObservableGauge<T>	Instantaneous value, read via callback	Runtime metrics, AddRuntimeInstrumentation(): GC heap size, thread count

A Counter and a Span can start in the same breath, so they can't drift apart. Bookmarks.Api's ServiceOperation helper starts both together, and records the Histogram when the operation's Dispose() runs:

```
using var operation = ServiceOperation.Start(ActivityNames.CreateBookmark);
⋮
ApplicationDiagnostics.BookmarksCreatedCounter.Add(1);
```

Exporting to Prometheus

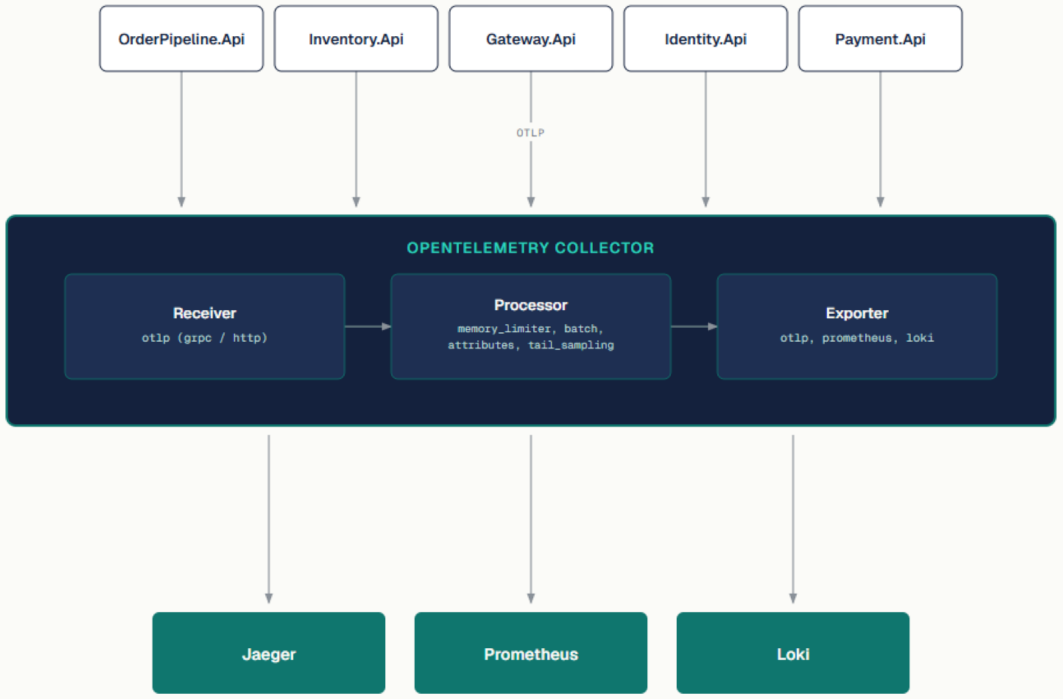
The Collector's prometheus exporter turns orders.created into orders_created_total, Prometheus's own naming convention for a Counter, and serves it for Prometheus to scrape.



The step from 1 to 2 is two real orders placed a few minutes apart, not a synthetic reading. exported_job is the same Resource attribute from Manual Instrumentation, carried all the way through.

One Pipe, Not One Vendor

CloudWatch Logs and Application Insights tie your instrumentation code to one vendor's SDK. With OpenTelemetry, every service only needs to know one address: the **Collector's**. The Collector decides where telemetry goes next, Jaeger, Prometheus, Loki, without the service's code ever changing.



EVERY SERVICE KNOWS ONE ADDRESS. SWAP A BACKEND BY EDITING THE COLLECTOR, NEVER THE SERVICES.

You Don't Always Need One

Bookmarks ships with no Collector at all. Its own README states the decision plainly, and it's the same decision you'd make for any service:

Situation	Collector?	This talk
One service, one backend that speaks OTLP	No	Bookmarks → straight to Elastic Cloud
A backend that can't accept OTLP (Prometheus, Loki)	Yes	OrderPipeline → Jaeger / Prometheus / Loki
Fanning the same telemetry to 2+ backends	Yes	OrderPipeline → 3 backends at once
One egress point, one credential, for many services	Yes	OrderPipeline's 5 services, 1 endpoint

For Bookmarks, a Collector would receive OTLP and forward the exact same OTLP, unchanged. That's not a pipeline, it's a hop with nothing to decide. Add one when a row above becomes true, not before.

Receiver, Processor, Exporter: Nothing Else

Every Collector config is the same three stages, wired into a pipeline per signal. This is the real, running config:

```
receivers: { otlp: { protocols: { grpc:, http: } } }
processors: { memory_limiter:, attributes:, tail_sampling:, batch: }
exporters: { debug:, otlp: {endpoint: jaeger}, prometheus:, loki: }
service.pipelines.traces: { receivers: [otlp], processors: […, tail_sampling, batch], exporters: [otlp] }
```

Every service points at the same Collector address, nothing else:

```
OTEL_EXPORTER_OTLP_ENDPOINT: "http://otel-collector:4317"
```

Same environment variable, every service, every signal. Switching Jaeger for Elastic APM is one exporter block in this file, not a redeploy of 5 services.

OpenTelemetry Doesn't Replace Your Logger

A Trace tells you what happened and where the time went. A Log tells you what the code said, in its own words, at the moment it happened. Neither one does the other's job. Adding OpenTelemetry's logging bridge adds a second export path alongside ILogger, it does not remove the console logger a platform like CloudWatch already reads.

Bookmarks.Api even has a second, deliberately non-OpenTelemetry logging path, to make that point concrete:

Shared.Observability/AuditLoggerProvider.cs

```
// Stamps Activity.Current?.TraceId onto plain stdout, bypassing log-level filtering.
// Correlation with a Trace doesn't require routing every log through OpenTelemetry.
```

The TraceId is just a string. Anything that can read Activity.Current can attach it, with or without the OTel SDK in the loop.

Configuring and Exporting Logs: Two Separate Calls

.WithLogging(...) takes two delegates on purpose: one configures the export pipeline, the other configures how ILogger itself behaves.

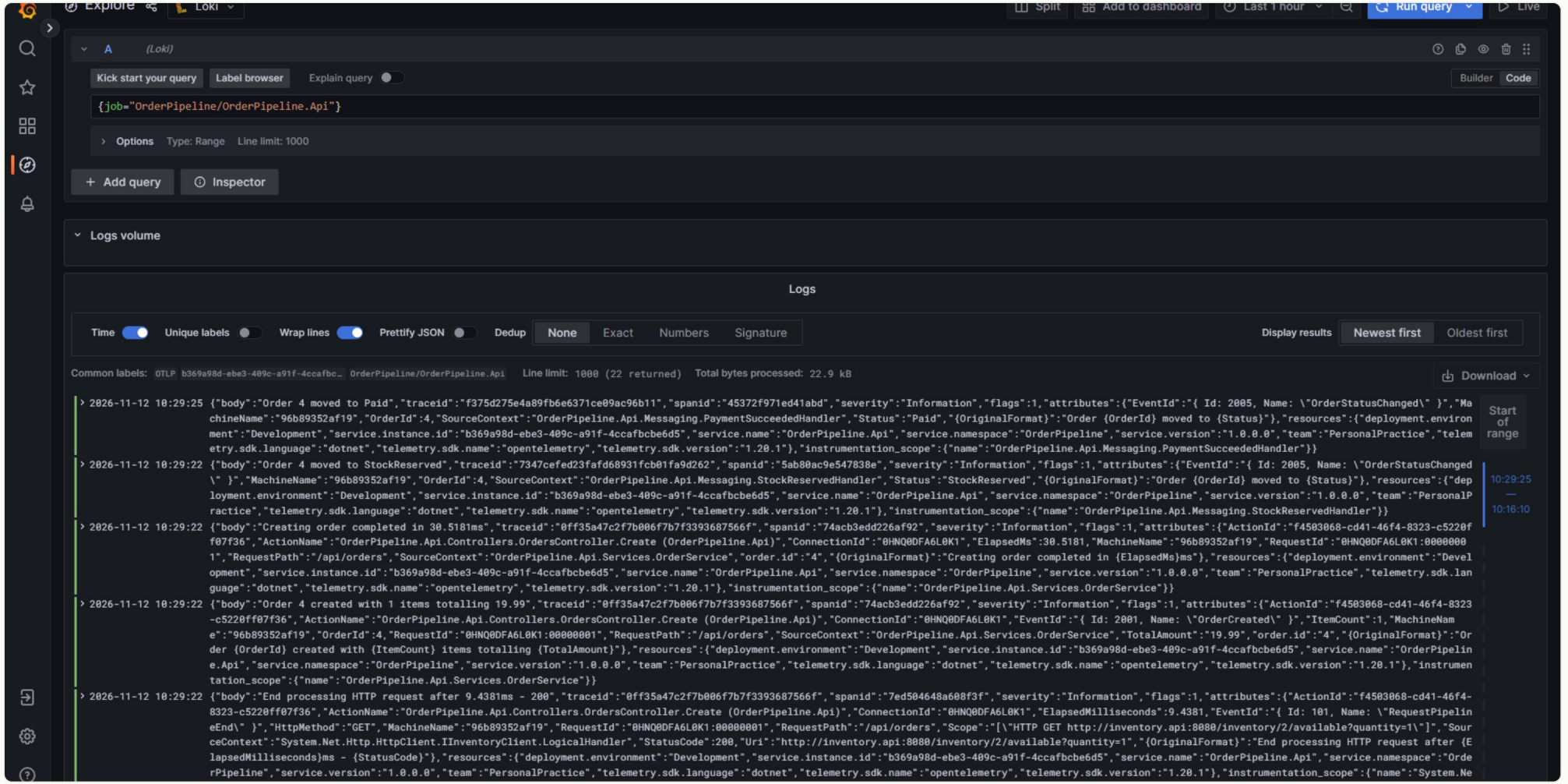
Shared.Observability/Internal/LoggingConfigurator.cs

```
public static void Configure(LoggerProviderBuilder logging, ObservabilityOptions options)
{
  if (options.Export.Console) logging.AddConsoleExporter();
  if (options.Export.Otlp is true) logging.AddOtlpExporter(…);
}
public static void ConfigureLoggerOptions(OpenTelemetryLoggerOptions o)
{
  o.IncludeFormattedMessage = true;
  o.IncludeScopes = true;
}
```

Without IncludeFormattedMessage, Loki stores your template string and the arguments separately, never the sentence you actually logged.

Exporting to Loki

A real {job="OrderPipeline/OrderPipeline.Api"} query in Grafana, right after placing an order. Every line already carries traceid, spanid, and the exact message, "Order 4 created with 1 items totalling 19.99", with no string-parsing to extract it.



Every line in this list shares the same traceid as the Span that was active when it was logged. The same Trace you'd see as a timeline in Jaeger, read back out here as text.

Logs Get the Same Enrichment as Spans, From the Same Source

Bookmarks registers two Processors from one allow-list of Baggage keys, one for Spans, one for Logs. Neither one invents new data, both copy whatever the caller already set:

BaggageTagProcessor : BaseProcessor<Activity>

```
activity.SetTag(entry.Key, Truncate(entry.Value));
```

BaggageLogProcessor : BaseProcessor<LogRecord>

```
logRecord.Attributes = merged;
```

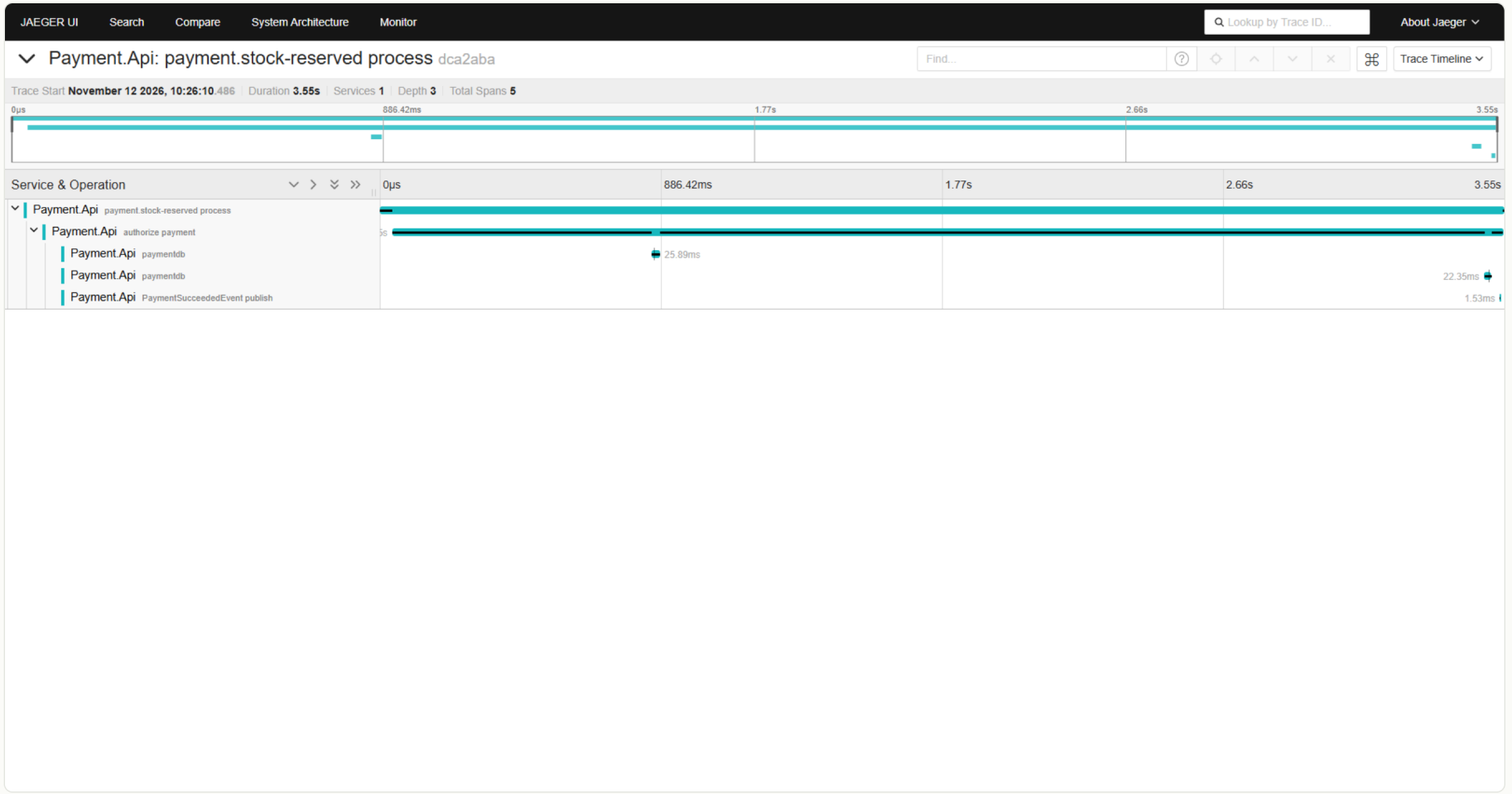
Both read the same AllowedKeyPrefixes option, so a Span and the Log line written a moment later carry the identical business context:

```
"Baggage": { "AllowedKeyPrefixes": [ "app." ] }
```

HTTP Propagates Itself. RabbitMQ Doesn't, by Default.

HTTP Instrumentation Libraries inject and read traceparent for you. A message queue has none, so OrderPipeline writes Propagator.Inject before BasicPublish, and Propagator.Extract before dispatching a consumed message, by hand, on both ends.

Checked live, against this running stack: even with that code in place, a fresh checkout's Trace still breaks at both RabbitMQ hops. The HTTP path correlates perfectly. The async path, right now, does not.



The lesson isn't "RabbitMQ propagation is easy." It's: Inject/Extract code existing is not the same as it working. Always check the real Trace in your own backend before you claim it does.

Baggage: the Same W3C Header, Two Very Different Senders

Baggage is key-value business context that rides inside the standard baggage HTTP header, separate from traceparent. Whoever reads it decides what to do with it, which is why it works even where there's no OpenTelemetry SDK at all.

Bookmarks: a browser, no SDK

```
// web/src/api/client.ts
headers.set("baggage",
  `app.action=${action},app.screen=${screen}`);
```

One plain fetch header. No package, no exporter, no credentials in the browser.

OrderPipeline: service to service

```
// OrderPipeline.Api/Services/OrderService.cs
Baggage.SetBaggage(OrderPipelineTags.OrderId,
  order.Id.ToString());
```

Set once in .NET, it then travels automatically over every outgoing HTTP call made from that request.

Either way, nobody writes code to *read* it at the far end. Shared.Observability's BaggageTagProcessor copies whichever keys are allow-listed onto every Span, automatically, in both projects.

Baggage Goes Everywhere the Request Goes, Including Out

Baggage attaches to every outgoing call after it's set, not just calls between your own services. If that same request later calls a third-party API, the same Baggage goes there too. It is, by design, untrusted input from whoever set it.

Both projects guard the same way: an allow-list by prefix, applied before anything is copied onto a Span or a Log, with hard caps on how much gets through.

Shared.Observability/Options/ObservabilityBaggageOptions.cs

```
public IList<string> AllowedKeyPrefixes { get; } = [];
public int MaxEntries { get; set; } = 16;
public int MaxValueLength { get; set; } = 256;
```

With no prefixes configured, the Processor copies nothing, that's the default. Bookmarks allows only app.. Never put a customer name, email or token behind an allowed prefix, only identifiers you'd be comfortable finding in a Span years from now.

A Span Processor Sees Every Span, Before Export

A **Span Processor** is a hook the SDK calls on every Span's start and end, before an Exporter ever sees it. It's how Baggage enrichment from the Context Propagation section is actually implemented, not a special case, just a Processor you can write yourself.

Shared.Observability/BaggageTagProcessor.cs, in full

```
public sealed class BaggageTagProcessor : BaseProcessor<Activity>
{
  public override void OnEnd(Activity data)
  {
    if (_options.AllowedKeyPrefixes.Count > 0) CopyAllowedBaggage(data);
    base.OnEnd(data);
  }
}
```

Registered the same way an exporter is, order matters, a Processor added after the exporter runs too late to change what gets exported:

```
tracing.AddProcessor(new BaggageTagProcessor(options.Baggage));
```

A Failure Is a Status, Not Just a Log Line

An unhandled exception should never disappear from a Trace silently. RabbitMqConsumer marks the Span, records the exception as a Span event, and only then logs and dead-letters the message:

Shared.Messaging/RabbitMqConsumer.cs, catch block

```
activity?.SetStatus(ActivityStatusCode.Error, ex.Message);
activity?.AddException(ex);
channel.BasicNack(@event.DeliveryTag, multiple: false, requeue: false);
```

This is also exactly what the Collector's tail_sampling policy, "keep-errors", checks for: a Span's status_code. Without SetStatus(Error), that policy has nothing to match.

Span Links connect two Spans that aren't parent and child, for example a batch job's Span linking back to every order it processed. Neither project needed one, so there's no real citation here: the shape is ActivitySource.StartActivity(name, kind, links: new[] { new ActivityLink(otherContext) }).

Keeping Every Trace Is a Cost Decision, Not a Default

Every Span held in Jaeger, every log line held in Loki, costs storage and, on a paid backend, money. **Sampling** is the deliberate decision to keep a subset of Traces and still have enough signal to answer the questions this talk opened with.

This isn't theoretical here: this stack's own Collector runs a tail_sampling policy that keeps every error but only half of everything else, which is exactly why some of the earlier screenshots in this talk took two tries to capture.

Two different places make this decision, and they don't make it the same way.

Head Decides Blind. Tail Decides Having Seen Everything.

Head Sampling, in the SDK

Decided at the first Span, before anyone knows if this Trace will contain an error. Cheap: nothing dropped later needed exporting in the first place.

```
// Shared.Observability/Internal/ObservabilitySamplerFactory.cs
SamplingMode.ParentBasedTraceIdRatio ⇒
  new ParentBasedSampler(new TraceIdRatioBasedSampler(ratio))
```

Tail Sampling, in the Collector

Decided after the whole Trace has arrived and waited out decision_wait. Can keep every error, because by then it knows which Traces have one.

```
# deploy/otel-config.yaml, real, running config
tail_sampling: { decision_wait: 10s, policies: [
  {name: keep-errors, type: status_code, codes: [ERROR]},
  {name: sample-the-rest, type: probabilistic, pct: 50}] }
```

They interact one way, never the other: Head runs first. If it drops a Trace, that Trace never reaches the Collector, so Tail never gets a chance to keep it either, even if it would have contained an error.

A Custom Sampler Is a Switch Statement, Not a New Concept

The SDK needs one object implementing Sampler. Shared.Observability doesn't write a new sampling algorithm, it picks between the SDK's own built-in Samplers from one config value, which is the shape most real services actually need:

Shared.Observability/Internal/ObservabilitySamplerFactory.cs, in full

```
public static Sampler? Create(ObservabilitySamplingOptions sampling)
  => sampling.Mode switch
  {
    SamplingMode.AlwaysOn => new AlwaysOnSampler(),
    SamplingMode.TraceIdRatio => new TraceIdRatioBasedSampler(sampling.Ratio),
    _ => null // SamplingMode.Default: leave OTEL_TRACES_SAMPLER alone
  };
```

Returning null for the default case is deliberate: it means "don't call SetSampler at all," so the standard OTEL_TRACES_SAMPLER environment variable still works for anyone who sets it, rather than being silently overridden.

What Changes When You Add This

Serilog / CloudWatch / App Insights alone	+ OpenTelemetry
A log line per event	+ a Trace connecting every event in one request, across services
A dashboard built for questions asked in advance	+ Metrics and Traces you can explore for questions nobody wrote a dashboard for
One vendor's SDK in your code	+ one Collector address; the backend becomes a config change
A correlation id threaded through calls by hand	+ TraceId and Baggage that travel with the request automatically

Every claim in this talk was checked against two real, running projects, including the one that didn't hold up on a second look: the checkout saga's RabbitMQ hops. That's the part worth remembering most, this only works if you keep checking it.

Source code for every example is shared separately.

APPENDIX: LOGGING FUNDAMENTALS

Logging in .NET

The foundation underneath everything shown so far: from ILogger basics to Serilog, performance, and monitoring, on the same OrderPipeline.Api.

The Foundation, One Topic at a Time

Topic	Focus
What is Logging?	ILogger, category, level, provider
The modern .NET Logger	DI, structured logging, event IDs
The logging packages	Generic Host, filters, exceptions
What is a provider?	App Insights, custom providers
Message template formatting	Timing, scopes, runtime levels
What is Serilog?	Sinks, enrichment, masking
Why does it matter?	Performance, source generators
What is monitoring?	Alerts, logging vs. monitoring

Four Building Blocks, Six Levels

ILogger
What you call to write a message.

Category
Which part of the app, the full type name by default.

Level
A severity scale used to filter what gets written.

Provider
Where the log goes: Console, Application Insights.



Each level is a filter threshold, lowest to highest. LogLevel.None only turns logging off in configuration, it is not a seventh level.

DI Gives You ILogger; Named Placeholders Make It Queryable

Constructor injection

```
public OrdersController(
  ILogger<OrdersController> logger)
```

The Host builds the ILoggerFactory once, at startup. Your class just asks for it.

String interpolation: don't

```
logger.LogInformation(
  $"Order {orderId} created");
```

One flat string. Nothing left to filter or group by.

Named placeholder: do

```
logger.LogInformation(
  "Order {OrderId} created", orderId);
```

Stored as OrderId = 1023: one queryable field.

Category and Event ID

Category follows the namespace

```
namespace OrderPipeline.Api.Controllers;
ILogger<OrdersController>
// category: OrderPipeline.Api.Controllers.OrdersController
```

Event ID names a kind of event

```
public static class LogEvents
{
  public const int OrderCreated = 2001;
}
```

A named constant, not a magic number: one Event ID per kind of event, independent of level and category.

Configuring Logging: the Host, and Its Filters

Both live in the same configuration layer: the Host decides how logging starts up, filters decide what each provider actually writes.

Generic Host

```
await Host.CreateDefaultBuilder(args)
  .ConfigureLogging(l ⇒ l.AddFilter("System", LogLevel.Debug))
  .Build().RunAsync();
```

Owns configuration, DI, and graceful shutdown. ASP.NET Core is itself a Generic Host.

Filtering by category

```
"Console": { "LogLevel": {
  "Microsoft.AspNetCore": "Information"
} }
```

When rules overlap, the most specific category wins. Not "code beats config," not "last one registered."

A Provider Is Two Methods; Application Insights Plugs Into Either

ILoggerProvider / ILogger

```
CreateLogger(string): ILogger
```

```
Log<TState>(level, eventId, state,
exception, formatter)
```

Every LogInformation/LogWarning call funnels into Log<TState> underneath. Application Insights ships a provider implementing exactly this interface; only the registration differs by host:

ASP.NET Core

```
builder.Logging.AddApplicationInsights(...);
```

Console app

```
services.AddLogging(x ⇒
x.AddApplicationInsights(...));
```

No built-in DI container. Build one by hand, flush before exiting.

Timing and Context, Without Repeating Yourself

Timing: IDisposable, not Stop()

```
using var timer = logger.BeginTimedOperation(
  "Creating order for {CustomerName}", name);
```

Dispose() always runs, even on an exception. No Stop() call to forget.

Scopes carry context

```
using (logger.BeginScope(new Dictionary<string, object>
  { ["OrderId"] = orderId })) { /* every log carries OrderId */ }
```

No parameter to repeat on every call inside the block.

Runtime levels: ASP.NET Core watches appsettings.json by default, edit the minimum level while running, save, and it applies immediately. appsettings.Development.json overrides the base file locally.

Sink Is Serilog's Word for Provider

UseSerilog(): keeps ILogger<T>

```
builder.Host.UseSerilog((ctx, _, cfg) ⇒ cfg
  .ReadFrom.Configuration(ctx.Configuration)
  .WriteTo.Console());
```

OrdersController still calls _logger.LogInformation(...). Nothing changes.

AddSingleton(logger): Serilog.ILogger

```
builder.Services.AddSingleton(logger);
app.MapGet("/", (Serilog.ILogger log) ⇒
  log.Information("Hello!"));
```

Every call site must switch to .Information(), .Warning()...

For an existing codebase like OrderPipeline.Api, UseSerilog() is the obvious choice.

Configuration and Enrichment

Its own config section

```
"Serilog": {
  "MinimumLevel": { "Default": "Information" },
  "WriteTo": [ { "Name": "Console" } ],
  "Enrich": [ "FromLogContext", "WithMachineName" ],
  "Properties": { "Application": "LoggingCourse" }
}
```

A section of its own, separate from "Logging", not shared with it.

Dynamic vs. static

FromLogContext: picks up properties pushed in code

WithMachineName / WithThreadId: recomputed on every log

Properties: one fixed value for the app's whole lifetime

Destructuring and Masking

{@Object} really destructures here

```
[LogMasked(ShowFirst = 3, PreserveLength = true)]
public string CustomerEmail { get; set; }

logger.Information("Order {@Order}", order);
```

Requires .Destructure.UsingAttributes() to take effect.

"an.nguyen@example.com"

↓

an.**********

First 3 characters kept, rest masked, original length preserved.

A Custom Sink Is One Method

```
public class OrderConsoleSink : ILogEventSink
{
  public void Emit(LogEvent logEvent)
  {
    var message = logEvent.RenderMessage();
    Console.WriteLine($"{DateTime.UtcNow} - {message}");
  }
}
```

5 → 1

methods to implement: ILoggerProvider + ILogger
vs. ILogEventSink

The Real Cost, and the Fix

Three real costs, every call

- Parsing the message template
- Boxing value-type arguments into object[]
- Allocating a new array

[LoggerMessage]: matches by name

```
[LoggerMessage(EventId = 2001,
  Level = LogLevel.Information,
  Message = "Order {OrderId} for {CustomerName}")]
public static partial void OrderCreated(
  this ILogger logger, int orderId, string customerName);
```

Parameter order can differ from the template. The generator matches names, not positions.

Error-Based vs. Query-Based Alerts

Error-based

A built-in signal (Exceptions, Failed Requests) plus a simple threshold. Fast to set up, but blind to anything that isn't an exception.

Query-based (KQL)

A custom query over any field: category, EventId, a structured property. Catches spikes that never throw an exception.

Window	OrderNotFound (Warning)	Exceptions
Normal 5 minutes	3	0
Incident 5 minutes	500	0

An error-based alert never fires here. There is no exception in either row. Only a query over the EventId catches the spike.

OrderPipeline.Api: Everything, Applied

An ASP.NET Core + PostgreSQL Order Management API, built up one topic at a time.



Cross-cutting, wraps every layer

Serilog pipeline

OrderAuditLoggerProvider

[LoggerMessage]

No Microservices? OpenTelemetry Still Applies

Serilog is the foundation: structured, fast logging in one process. Everything shown earlier about Tracing, Metrics, and correlation works the same way in a single standalone project, with no second service required.

In one process, you still get	Why it's worth it even alone
A Span around a slow internal operation	See exactly which step of a background job or query took the time, not just that it was slow overall
A Counter or Histogram	Cheap, aggregated numbers instead of mining your own logs for trends
Logs correlated to a request's TraceId	Useful even within one process once more than one background task runs concurrently
Vendor-neutral export from day one	If this project ever does grow into more services, nothing about the instrumentation changes

The exact same AddOpenTelemetry().WithTracing(...) shown throughout this talk is the whole setup. Microservices were never the requirement.

Thank You